

How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior

Zidi Xiong^{1*}, Yuping Lin^{3*}, Wenya Xie⁴, Pengfei He³,
Jiliang Tang³, Himabindu Lakkaraju¹, Zhen Xiang²

¹Harvard University ²University of Georgia

³Michigan State University ⁴University of Minnesota-Twin Cities

Abstract

Memory is a critical component in large language model (LLM)-based agents, enabling them to store and retrieve past executions to improve task performance over time. In this paper, we conduct an empirical study on how memory management choices impact the LLM agents’ behavior, especially their long-term performance. Specifically, we focus on two fundamental memory operations that are widely used by many agent frameworks—*addition*, which incorporates new experiences into the memory base, and *deletion*, which selectively removes past experiences—to systematically study their impact on the agent behavior. Through our quantitative analysis, we find that LLM agents display an *experience-following property*: high similarity between a task input and the input in a retrieved memory record often results in highly similar agent outputs. Our analysis further reveals two significant challenges associated with this property: *error propagation*, where inaccuracies in past experiences compound and degrade future performance, and *misaligned experience replay*, where outdated or irrelevant experiences negatively influence current tasks. Through controlled experiments, we show that combining selective addition and deletion strategies can help mitigate these negative effects, yielding an average absolute performance gain of 10% compared to naive memory growth. Furthermore, we highlight how memory management choices affect agents’ behavior under challenging conditions such as task distribution shifts and constrained memory resources. Our findings offer insights into the behavioral dynamics of LLM agent memory systems and provide practical guidance for designing memory components that support robust, long-term agent performance. We also release our code to facilitate further study.²

1 Introduction

Recently, advances in Large Language Models (LLMs) with strong instruction-following capabilities have paved the way for LLM agents [22, 30, 25] – an autonomous system powered by LLM that can interact with the external environment and perform tasks. Unlike stand-alone LLMs, LLM agents possess the ability to perceive the environment, engage in step-by-step reasoning and planning, evolve through memory, and make decisions to solve complex real-world tasks [24, 17, 21, 19, 27].

Among the various components, memory plays a pivotal role in LLM agent execution [22]. It retains past task queries (i.e. the agent input) and execution (i.e. the agent output), which can be retrieved as in-context demonstrations to guide similar future tasks. Through effective memory management, including adding, updating, and deleting past experiences, the performance of LLM

*Equal contributions. Correspondence to: Zidi Xiong zidixiong@g.harvard.edu and Zhen Xiang zhen.xiang.lance@gmail.com

²The dataset and experimental code are available in https://github.com/yuplin2333/agent_memory_manage.git

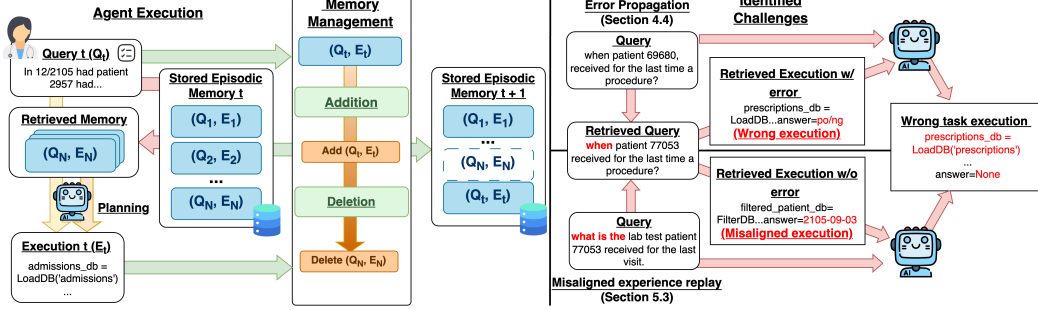


Figure 1: **Left:** Illustration of the memory management workflow after each agent execution. **Right:** identified challenges along with experience following, including **error propagation** (Section 4.4): errors in previously stored executions propagate to similar tasks when being retrieved; and **misaligned experience replay** (Section 5.3): similar but inherently misaligned records weaken experience-following effectiveness.

agents can be improved over time [30]. However, despite the importance, the properties of the general memory module for LLM agents and their management remain largely under-explored. While some studies have investigated effective memory management for specific agent types [28, 29, 31, 32, 23], they fail to provide a broader understanding that could inform the development of better memory systems for a wide range of autonomous LLM agents. The major obstacle to an in-depth study of the memory module lies in the diversity of agents’ task objectives and functionalities, which leads to highly varied—and often inconsistent—memory designs [18, 13, 21, 11, 16, 26]. For example, EHRAgent [18] stores generated code snippets for tasks related to electronic health records (EHRs), whereas AgentDriver [13] maintains a vector encoding past states and corresponding trajectory prediction by the agent.

To bridge this gap, we present an in-depth quantitative study to uncover the fundamental principles of memory management for general autonomous LLM agents. To draw broadly applicable insights across diverse memory designs, we focus on two essential memory operations—*addition* and *deletion*—which are widely adopted by existing agents, as illustrated in Figure 1. We quantitatively investigate how different memory addition and deletion designs shape agent behavior and identify an important phenomenon that we term the *experience-following* property: a high ‘input similarity’ between the current task query and the one from the retrieved record often yields a high ‘output similarity’ between their corresponding (output) executions.

While the experience-following property supports effective reuse of successful experiences, we also uncovered two notable challenges associated with it. First, we observe the problem of *error propagation*: if a retrieved memory record contains low-quality or incorrect outputs, the agent is likely to replicate and even amplify these errors during the current task. If the resulting execution is then added back into memory, the error is likely to be further propagated to future tasks (Figure 1). Second, we recognize the issue of *misaligned experience replay*, which limits the benefits of experience following—certain memory records, when retrieved as demonstrations, consistently result in low output similarity, showing their inadequacy as demonstrations. Retaining these records increases the likelihood of suboptimal or incorrect executions (Figure 1).

Through controlled experiments on three agents across diverse tasks, we systematically analyze how selective strategies for the addition and deletion of memory entries can influence long-term agent behavior. Our findings show that to mitigate error propagation, a *Selective Addition* approach should be employed, where each task query and its associated output execution are assessed for quality insurance before being integrated into the memory. Furthermore, to counter misaligned experience replay and reduce memory size for more efficient storage, we propose a simple *Combined Deletion* strategy: redundant records will be deleted based on recent retrieval frequencies, while low-quality and outdated records will be deleted based on their historical task utilities as demonstrations.

We further validate our observations under challenging conditions representing real-world scenarios, including (1) *Task Distribution Shift*: the task distribution changes substantially over time, requiring the agent to adapt to shifting patterns and contexts. (2) *Memory Resources Constraint*: the memory capacity is severely limited, requiring the agent to retain only the most valuable and helpful experience.

The results underscore the importance of thoughtful memory management for maintaining stable and effective long-term agent performance. Our contributions are summarized below:

- We conduct systematic and quantitative analysis of the impact of the two most essential memory operations—addition and deletion—of the memory records on the long-term performance of LLM agents. Our findings provide key principles for robust memory management design.
- We reveal an experience-following property for LLM agents with memory and highlight two important challenges for memory management—error propagation and misaligned experience replay—arising with this property.
- We propose and advocate a general framework incorporating a selective addition and a combined deletion policy. Through the evaluation of three agents for three different tasks, respectively, we show the effectiveness of this framework in mitigating both error propagation and misaligned experience replay, with an average 10% absolute performance gain over the baseline policies.
- We further validate the necessity of these policies under two demanding real-world conditions: task distribution shift and memory resource constraints. Our findings show that the combined policy helps agents adapt to evolving tasks and operate effectively with limited memory.

2 Background and Related Works

2.1 Memory Module of LLM Agents

LLM agents often include short-term memory and long-term memory [30]. Short-term memory usually refers to inside-task working memory [20], while long-term memory [20] can be divided into three types: *semantic memory*, *procedural memory*, and *episodic memory*. Semantic memory [9] contains the agent’s world knowledge and understanding of the environment; procedural memory [2] involves rules or procedures, which may reside implicitly in the LLM’s weights or be explicitly defined as guidelines for the agent; and episodic memory [15] records task-specific experiences. In this paper, we focus on episodic memory, which is commonly used by LLM agents with memory.

2.2 Memory Management and Limitations

LLM agents often employ an episodic memory module to store past experiences for future retrieval, which is crucial for effective planning and task execution [18, 13, 19, 4, 21]. The memory module typically includes operations such as memory reading and memory management [30]. Specifically, given a new task query q and a memory base currently containing N query-execution pairs $\mathcal{D} = \{(q_1, e_1), \dots, (q_N, e_N)\}$, the agent’s execution cycle involves the following memory operations:

Memory Reading: The agent retrieves a subset $\xi_K \subset \mathcal{D}$ consisting of the K query-execution pairs most relevant to the query q . The relevance is often measured by the input similarity between the task query q and the query from the retrieved past experience. For example, the relevance can be computed as the cosine similarity between the feature representations for both queries from a text encoder [10]. The retrieved pairs ξ_K are then used as in-context learning demonstrations to guide the LLM in generating an execution trajectory e for the task query q .

Memory Management and Limitations: Memory management typically involves the *addition* and *deletion* of memory records. Specifically, when obtaining the query-execution trajectory pair (q, e) , addition decides whether this pair should be added to memory, while deletion [32, 12] is often triggered to remove outdated or redundant pairs from memory.

Although various strategies have been proposed for memory management, such as structural transformation [21, 1, 29], merging [32, 12, 6], summarization [23, 32], and reflection [19, 31], these approaches are often designed for specific agent types (e.g., chatbot agents) and do not provide a unified set of principles. Parallel works also conduct quantitative evaluations of different memory management methods, including a structural transformation and retrieval method [29] and an expectation maximization approach for optimizing the memory bank [28]. However, their findings remain tailored to certain agent types and are difficult to extend to more complex LLM agents, such as code agents covered by our study [28].

In contrast, our work centers on the basic operations of memory addition and deletion, aiming to uncover how these basic memory operations influence the behavior and long-term performance of

generic LLM agents. Our findings will shed light on potential challenges posed by these operations and offer insights to guide future memory system design.

3 Evaluation Setup

In this paper, we investigate memory management in generic LLM agent settings by considering three representative agents designed for different tasks: EHRAgent [18], AgentDriver [13], and CIC-IoT Agent [14]. In addition to their distinct tasks, these agents also vary significantly in input-output formats and memory retrieval mechanisms, which enhances the generalizability of our findings. See Table 3 in Appendix A.1 for a high-level summary of these agents.

EHRAgent [18]: EHRAgent is a code-generation agent that allows clinicians to interact with electronic health records (EHRs) using natural language queries. We adopt the MIMIC-III dataset [8] for evaluation. By filtering out duplicated data and those lacking answers, we get 2392 tasks in total. Following the original study, we retrieve four past experiences during the execution of each test query based on a general text embedding model and maximum cosine similarity. The initial memory bank contains 100 records. The accuracy (ACC) is reported as task performance.

AgentDriver [13] AgentDriver is an LLM-based autonomous driving agent that integrates common sense and experience into the memory record. Following the original setup, we use the nuScenes dataset [3] and randomly sample 2000 test cases from its test set. We slightly simplify the original two-step retrieval of AgentDriver to a single-step retrieval using top-1 vector similarity, i.e., the retrieval results in a single demonstration (see Appendix A.2 for details). Task performance is evaluated using the success rate (SR), which measures the proportion of executions that predict a trajectory with an average L2 error below 2.5. Here, the L2 error is measured in UniAD [7] over 3-second, which quantifies the distance between the predicted and ground-truth trajectories. For the initial memory, we randomly sample 180 experiences from the nuScenes training set.

CIC-IoT Agent CIC-IoT Agent is a network intrusion detection agent designed to predict attack types based on IoT packet features. Specifically, we construct prompts by formatting these features according to a predefined template and input them into the Agent, which then predicts the corresponding attack type. We evaluate CIC-IoT Agent using the CIC-IoT benchmark dataset [14]. Given the large scale of the dataset, we randomly sample 1200 test cases from the test set. For each test query, we retrieve three past experiences during execution using a feature-based approach, as detailed in Appendix A.3. To initialize the memory bank, we use GPT-4o to generate 100 records from a disjoint training set. Task performance is measured by accuracy (ACC)

For all three agents, we use GPT-4o-mini as the backbone language model for most of the experiments. More details can be found in Appendix A.

4 Addition of Memory

Memory addition refers to the process of deciding whether a finished task execution should be stored as a new record in the memory bank. We investigate three memory addition strategies—an add-all baseline, selective addition based on coarse automatic evaluation, and selective addition based on strict human evaluation—along with a fixed-memory baseline without memory addition. Through our empirical evaluation of these strategies, we have three central findings: **(1) Selection is the key to effective memory addition:** Selective addition based on strict human evaluation consistently surpasses the fixed-memory baseline and the add-all strategy without a memory filter (Section 4.2). **(2) Agent execution exhibits an experience-following property:** When the retrieved task query closely resembles the current task, the input similarity often strongly correlates with the similarity of their output executions (Section 4.3). **(3) Error propagation:** Through an in-depth investigation of the experience-following property, we identify an *error propagation* challenge for memory management and demonstrate how selective addition helps mitigate this issue (Section 4.4).

4.1 Setup for Memory Addition Experiments

We begin by describing several memory addition strategies. For a query-execution pair (q, e) and an evaluator π , the addition decision is given by $\pi(q, e)$: if $\pi(q, e) = 1$, the experience is stored; if $\pi(q, e) = 0$, it is discarded. Our aim is to analyze how different addition strategies influence the

agent’s behavior and performance in the long run. Starting from an identical initial memory, we examine the following four memory addition strategies:

1) Fixed-memory baseline: To establish a baseline and demonstrate the need for memory addition, we first consider a fixed memory bank. In this setting, we use a subset of the training data with ground truth agent execution as the fixed memory bank [13, 11, 31], and the agent relies only on this fixed memory without adding new entries, i.e., $\pi_{\text{fixed}}(q, e) = 0$.

2) Add-all approach: A straightforward baseline is to store every encountered task and its execution: $\pi_{\text{all}}(q, e) = 1$.

3) Selective addition based on automatic evaluation (coarse): Coarse selective addition employs an LLM to decide whether an execution should be added to the memory, i.e., $\pi_{\text{automatic}}(q, e) = \text{LLM}(q, e)$. This method is broadly applicable and relatively cost-effective. Detailed designs of these automatic evaluators are provided in Appendix A.4.

4) Selective addition based on human evaluation (strict): In this stricter approach, a human (oracle) serves as the evaluator, determining whether the execution should be stored in the memory: $\pi_{\text{human}}(q, e) = \text{Human}(q, e)$. Although human input could be gathered after each execution for practical agents, it is not feasible for our current evaluation. Therefore, we simulate this process by comparing the generated output with the ground truth.

Details on each agent’s specific designs for memory addition can be found in Appendix A.2.

4.2 Selection is Necessary for Memory Addition

We examine the performance of selective addition, add-all, and fixed-memory strategies. First, we compare their overall performance over the long run, and second, we analyze the trends in the performance and the underlying reasons.

Selective addition based on strict criteria outperforms all other strategies, while add-all is inferior to the fixed-memory baseline. Table 1 summarizes the performance of the three memory addition strategies on

the three agents, respectively. When comparing add-all with the fixed-memory baseline, we observe that simply storing every experience leads to significantly worse outcomes, particularly for EHRAgents and AgentDriver. In contrast, both coarse and strict selective addition maintain reasonable memory sizes while achieving higher performance in most cases compared with add-all.

Selective addition improves the long-term performance of agents through self-evolution, while add-all causes self-degradation.

Although the same initial memory is used by all methods in our evaluation, the long-term performances of these methods diverge considerably. Figure 2 and Figure 8 in Appendix B illustrate the accuracy trend for these three agents using each addition strategy. The curve for the add-all strategy either remains flat or declines over time, suggesting a probable accumulation of flawed entries that can degrade the agent’s performance. The curve for the fixed-memory baseline stays above the add-all approach but exhibits little improvement for EHRAgent and AgentDriver, as no examples could be further incorporated into the memory. In contrast, the performance for strict selective addition continues to improve over extended runs. This trend is also verified using other LLM backbones, as shown in

Table 1: Performance of the three memory addition strategies, add-all: selective addition based on automatic evaluation (coarse), and selective addition based on human evaluation (strict), compared with the fixed-memory baseline (shaded) on EHRAgents, AgentDriver, and CIC-IoT. The results highlight the necessity of selection for effective memory addition.

Strategy	EHRAgents		AgentDriver		CIC-IoT Agent	
	ACC $\uparrow$	Mem Size $\downarrow$	SR. $\uparrow$	Mem Size $\downarrow$	ACC. $\uparrow$	Mem Size $\downarrow$
Fixed	16.89	100	40.53	180	60.25	100
Add all	13.04	2411	32.48	2125	60.00	1300
Coarse	31.35	1881	37.03	1161	60.67	1083
Strict	38.86	1012	50.94	1178	63.33	860

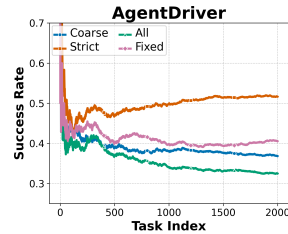


Figure 2: Performance trend over AgentDriver. Strict addition consistently improves the performance over time.

Appendix C.1 These results underscore the advantages of selectively integrating new experiences.

4.3 Experience-Following Property

For each memory addition strategy, we measure both the input similarities and output similarities between each query and the memory records retrieved during its execution. The computation of both types of similarities for each query can be found in Appendix A.2. For each of the input similarities and the output similarities, we compute the cumulative average over the entire sequence of test query executions to demonstrate the long-term trend. As shown on the left of Figure 3 for AgentDriver, fixed-memory baseline produces both low input and output similarities, whereas addition-based methods exhibit higher output similarity as the input similarity grows. Similar patterns are observed for the other two agents (Figure 9 of Appendix B) and alternative LLM backbones such as state-of-the-art GPT-4o and DeepSeek-V3 (Figure 15 of Appendix C.1). The exhibition of such a strong correlation between the input and output similarities is referred to as the **experience-following** property of memory-based LLM agents.

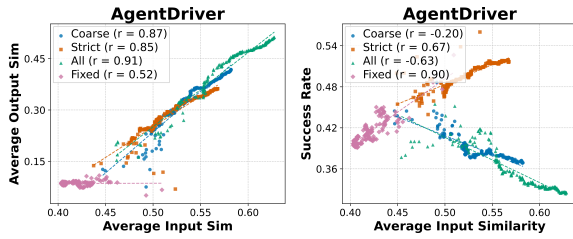


Figure 3: **Left:** Output similarity versus input similarity for AgentDriver over different addition strategies. **Right:** Average success rate versus input similarity of AgentDriver over different addition strategies.

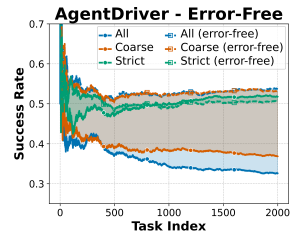


Figure 4: Comparison of running performance between using the agent output as demonstrations and the error-free variant using ground-truth.

Intuitively, the experience-following property reveals the fact that LLM agents tend to follow the retrieved memory records more strictly when their queries are highly similar to the current task query. As the memory base expands by incorporating more experience from diverse use cases, the agent will retrieve records resembling highly similar past experiences more easily. Therefore, a careful selection strategy for memory addition can facilitate the self-improvement of the agent through the replication of correct executions of similar tasks. Conversely, indiscriminate memory addition will easily introduce incorrect executions for the agent to learn from, causing a self-degradation of its performance. To visualize this phenomenon, on the right of Figure 3, we show the relationship between the cumulative average input similarity and the success rate for different memory addition strategies for AgentDriver. We observe that strict selective addition fosters a high average performance with high input similarity, whereas methods that incorporate noisy entries into the memory yield significantly lower performance even with high input similarity. Similar trends can also be found for other agents as shown in Figure 10 in Appendix B.

4.4 Error Propagation in Agent Memory

While experience-following enables LLM agents to learn from high-quality examples when selective memory addition is adopted, the addition of erroneous memory records remains inevitable in many cases even with human inspection of the agent execution. For example, in AgentDriver, there is always a discrepancy between the optimal trajectory and the one predicted by the agent. When erroneous memory records are retrieved as demonstrations, these errors can influence the current task execution. If the current execution is then stored in the memory, the error may propagate to future tasks by affecting their execution. This **error propagation** poses significant challenges to memory management, causing a deviation between the agent’s performance and the optimal level achievable with perfect memory.

In Figure 4, we visualize error propagation on AgentDriver. For each addition strategy in Section 4.1, we compare it with a variant that uses the same retrieved examples for each task but replaces the

LLM’s execution with the ground-truth trajectory, ensuring error-free³ retrieval. For each strategy, we observe an immediate gap in performance compared to its error-free variant. Moreover, as the execution continues, both add-all and coarse selective addition exacerbate such a performance gap, whereas strict selective addition, despite lagging initially, gradually approaches the ground-truth baseline and even surprisingly surpasses its performance after roughly 2000 executions. This behavior coincides with the high performance of AgentDriver using fixed, ground-truth memory without erroneous execution. However, when we add noise to the initial memory of AgentDriver (Appendix C.3), even with fixed memory, there is a rapid performance drop within 500 executions.

Interestingly, although with a much higher memory size, the performance of the error-free coarse addition strategy shows close performance to an error-free version of add-all. This observation naturally raises a question: Is retaining all memory records truly necessary to maintain a strong agent performance? This leads to the memory deletion strategies discussed in the sequel.

5 Deletion of Memory

Memory deletion is necessary for real systems, as the memory cannot grow indefinitely due to the practical constraint of storage. In this section, we evaluate three memory deletion approaches – periodical-based deletion, history-based deletion, and a deletion approach that comprehensively combines the two (Section 5.1). We present two key findings: **(1) Strategic memory deletion can further enhance the agent’s performance while reducing the memory size:** When used with selective addition, history-based memory deletion enhances the agent’s performance the most, while combined memory deletion reduces the memory size the most. (Section 5.2). **(2) Misaligned experience replay:** We identify *misaligned experience replay*, another challenge in memory management where some task-execution pairs are particularly difficult for the agent to benefit from experience-following. We show that the agent’s performance gain by history-based deletion can be attributed to the removal of the memory records associated with misaligned experience replay (Section 5.3).

5.1 Setup for Memory Deletion Experiments

Let (q_i, e_i) be a query-execution pair where $1 \leq i \leq N$ is the index. This pair will be removed from the memory bank if it satisfies a specific criterion ϕ . We focus on three memory deletion strategies that can be deployed together with selective memory addition (either coarse (automatic) or strict (human) evaluation):

1) Periodical-based Deletion: Prior research proposes to use a forgetting rate inspired by human cognition to assign deletion probabilities based on how frequently and how recently a memory record is retrieved [32, 5]. In this work, we adopt a simplified (thus more widely applicable) strategy using a fixed threshold to determine whether a memory record should be deleted based on its past retrieval frequency during a given period. Specifically, Let $\text{freq}_t(q_i, e_i)$ denote the total retrievals of (q_i, e_i) at the current timestamp t , and $\text{freq}_{t'}(q_i, e_i)$ be the retrievals at an earlier timestamp t' . Define α as the target retrieval count within $[t', t]$. A memory record will be deleted if $\phi_{\text{period}}(q_i, e_i, t, t') = 1$ where

$$\phi_{\text{period}}(q_i, e_i, t, t') = \mathbb{1}[\text{freq}_t(q_i, e_i) - \text{freq}_{t'}(q_i, e_i) \leq \alpha].$$

This approach keeps the memory size M bounded by $M \leq \alpha(t' - t)K$, where K is the number of retrieved memory per execution.

2) History-Based Deletion: We propose a history-based deletion strategy guided by the utility of stored memory records over time. Suppose Φ is a utility evaluator, which could be the same one used for selective addition. For any timestamp t , a record will be removed if a) it has been retrieved for at least n times, and b) the average utility across all its past retrievals is below a prescribed threshold β :

$$\phi_{\text{history}}(q_i, e_i, t) = \begin{cases} \mathbb{1}\left[\frac{1}{\text{freq}_t(q_i, e_i)} \sum_{m=1}^{\text{freq}_t(q_i, e_i)} \Phi(q_m, e_m) \leq \beta\right], & \text{if } \text{freq}_t(q_i, e_i) > n, \\ 0, & \text{otherwise.} \end{cases}$$

Note that we require a memory record to be retrieved at least n times before being considered for removal to reduce the estimation bias for the average utility. Moreover, a memory record with high

³While still suboptimal, the ground-truth trajectory resembles relatively “correct” task executions.

utility at timestamp t may still be removed in the future as the task distribution evolves. We will discuss this scenario in Section 6.1.

3) Combined Deletion: Periodical- and history-based methods can be applied together to jointly enhance the agent performance and reduce the memory size:

$$\phi_{combine}(q_i, e_i, t, t') = \phi_{period}(q_i, e_i, t, t') \times \phi_{history}(q_i, e_i, t)$$

5.2 Strategic Memory Deletion Improves the Agent Performance

Table 2 summarizes the effects of three deletion strategies—periodical-based, history-based, and combined deletion—on both agent performance and memory size.

We first observe that *periodical-based deletion* achieves substantial memory reduction with minimal performance degradation. Across most configurations, this method leads to absolute accuracy drops of less than 2%, indicating that addition-only memory designs often accumulate redundant entries.

History-based deletion and *combined deletion* show more variable results depending on the quality of the utility evaluator used during addition and deletion. Specifically, when a strict evaluator is employed, both strategies lead to notable performance improvements.

Table 2: Performance of periodical-based deletion, history-based deletion, and the combined approach, when deployed with the two selective addition strategies (strict and coarse) and using the same evaluator for history-based and combined deletion. History-based deletion enhances the agent’s performance in most cases, while the combined method reduces the memory size the most.

Strategy	EHRAgents		AgentDriver		CIC-IoT Agent	
	ACC. ↑	Mem Size ↓	SR. ↑	Mem Size ↓	ACC. ↑	Mem Size ↓
Coarse evaluator						
No del	31.35	1881	37.04	1161	60.67	1083
Period	30.30	379	36.39	426	59.58	163
History	32.04	1818	34.02	1019	53.08	1115
Combined	29.52	377	35.63	372	53.83	160
Strict evaluator						
No del	38.89	1012	50.94	1178	63.33	860
Period	39.04	302	50.94	467	61.83	148
History	42.65	784	51.81	846	67.67	755
Combined	42.88	248	49.81	323	63.50	82

To further understand these improvements, we extend the methodology used in Section 4.4 to construct an error-free memory baseline during agent execution. As presented in Appendix C.2, history-based deletion with a strict evaluator can even outperform this error-free counterpart with a larger magnitude compared with no deletion. This result suggests that selectively retaining helpful experiences observed during execution can lead to better long-term performance by adapting more effectively to evolving task demands.

However, when the evaluator used for both addition and deletion is coarse or noisy, deletion methods can negatively affect agent performance. This highlights the challenge of using a coarse evaluator—its inherent errors can propagate, particularly when it drives the selection criteria for history-based and combined deletion. Indeed, because the coarse evaluator relies on an LLM with limited task-specific knowledge, it can misjudge execution quality and sometimes remove beneficial examples. In practice, this drawback can be mitigated by fine-tuning the evaluator on specific tasks to reduce errors.

In summary, our findings show that with a reliable utility evaluator, history-based deletion can substantially improve agent performance. Additionally, the combined deletion approach offers a strong balance between maintaining performance and reducing memory size, making it a practical and efficient memory management strategy for long-term LLM agent deployment.

5.3 Misaligned Experience Replay

To explain the performance gains achieved by history-based deletion with a strict utility evaluator, we hypothesize that some memory records offer limited guidance during task execution. These records, which exhibit relative high input similarity to a task yet yield low output similarity when used as demonstrations, force the agent to rely more on its own reasoning—thus diminishing the benefits of experience following. We call this phenomenon *misaligned experience replay* to highlight the misalignment between the current task execution and the execution of its neighboring tasks retrieved from the memory bank as demonstrations. We attribute the performance gain of the history-based deletion to its successful removal of memory records associated with misaligned experience replay.

To visualize how history-based memory deletion handles misaligned experience replay, we divide a sequence of testing tasks into two groups: (1) *deleted group*: tasks for which the retrieved memory record exhibiting the highest output similarity has been deleted from the memory bank, ultimately and (2) *retained group*: the rest of the tasks with the top-output-similarity record retained in memory after all task executions. We then calculate the input and output similarity with the retrieved record that has the highest output similarity – we find clearly different patterns between the two groups.

Considering the tasks on EHRAgent, for example, in Figure 5, tasks in the deleted group demonstrate that deleted records have much lower output similarity than retained records despite having comparable input similarity. The highlighted regions of the figure reveal that these deleted records lack effective experience-following – even when the input similarity is high, they do not lead to closely matched outputs, which degrades the long-term agent performance. Similar results for the other two agents can be found in Figures 11 and 12 in Appendix B.

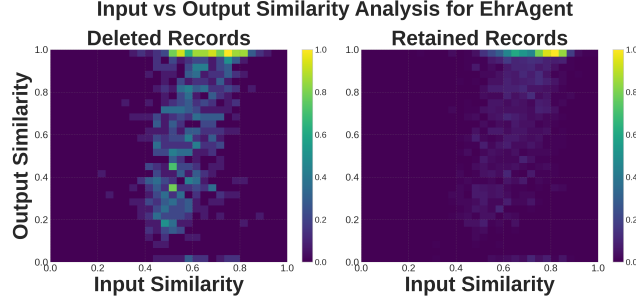


Figure 5: Correlation between input similarity and output similarity over tasks leveraging deleted records and those leveraging retained records. History-based deletion consistently removes poor demonstrations with low output similarity, thereby improving long-term performance.

6 Memory Management under Challenging Scenarios

6.1 Memory Management with Task distribution shift

Task distribution shift occurs when the predominant task type and the expected execution change over time. To simulate a task distribution shift, we construct a modified dataset from the original test sets of EHRAgent and AgentDriver. Specifically, we obtain the embedding vectors for all test queries using OpenAI text-embedding-3-large encoder for EHRAgent, and the original input vector computation method from AgentDriver. We then apply a Gaussian Mixture Model (GMM) to cluster the embeddings into three groups – each group will likely have a distinguished task distribution. Finally, we reorder the test queries based on their assigned cluster labels to process the three task groups sequentially. This procedure forces a pronounced shift in the task distribution.

Results In Figure 6, we compare performance trends across several memory configurations on EHRAgent and AgentDriver: fixed memory, strict addition, history-based deletion with a strict evaluator, and combined deletion with a strict evaluator. We also include results from a variant of combined deletion (with a strict evaluator) executed under a setting without task distribution shift, as shown in Section 5. We observe varying performance dynamics across task distributions. However, in general, the performance gap relative to the no-shift variant remains small. Notably, in AgentDriver, strict addition alone achieves performance that even surpasses the no-shift variant. In contrast, on EHRAgent, history-based deletion underperforms compared to combined deletion.

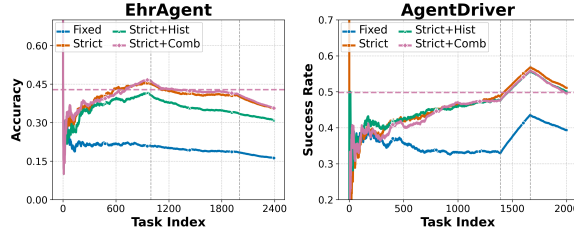


Figure 6: Performance comparison under task distribution shift for EHRAgent and AgentDriver. The vertical line indicates the point at which the task distribution shifts. The horizontal dashed line shows the performance of the combined deletion variant without distribution shift.

This suggests that in practical scenarios involving distribution shift, periodic deletion—despite its simplicity—can contribute to stabilizing performance.

6.2 Memory Management with Resource Constraints

We also investigate a scenario where the memory capacity is fixed, for example, to its initial size of 100 records for EHRAgent. Under this constraint, we modify the combined deletion policy so that after each task execution, it first performs periodical-based deletion and then removes only the record with the least average utility (rather than all records below the utility threshold) if the memory still exceeds the limit after additions.

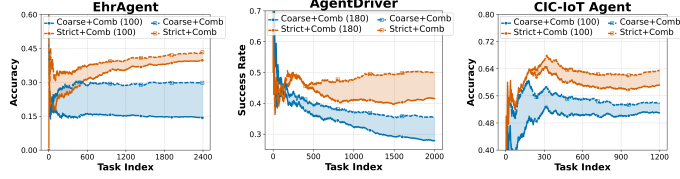


Figure 7: Comparison of unlimited memory size versus a limited memory size of 100 records for strict and coarse selective addition with combined deletion.

Results As shown in Figure 6, the memory management policies achieve high performance under strict capacity constraints compared with the fixed-memory variant. By selectively retaining only the most relevant and high-quality records, the agent utilizes limited storage efficiently. These results indicate that effective memory management choices can still improve the long-term agent performance in resource-limited settings. We also study the relationship between the size of the constrained memory and the performance on AgentDriver, as shown in Figure 18 in Appendix C.4, which demonstrates a gradually converging performance when using a strict utility evaluator. This also suggests that naive, unbounded memory growth is unnecessary.

7 Conclusion

This paper investigates memory management in general LLM agents through addition and deletion. We identify the experience-following phenomenon and its challenges: error propagation and misaligned experience replay. Experiments show that combining deletion with selective addition, guided by a strong evaluator, effectively mitigates these issues. The framework’s robustness and adaptability are further confirmed with two challenge scenarios.

References

- [1] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Mikhail Burtsev, and Evgeny Burnaev. Arigraph: Learning knowledge graph world models with episodic memory for llm agents. *arXiv preprint arXiv:2407.04363*, 2024.
- [2] H Beaunieux, V Hubert, T Witkowski, AL Pitel, S Rossi, JM Danion, B Desgranges, and F Eustache. Which processes are involved in cognitive procedural learning? *Memory*. 2006 Jul;14(5):521-39. doi: 10.1080/09658210500477766. PMID: 16754239., 2006.
- [3] Holger Caesar, Varun Bankiti, Alex H Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11621–11631, 2020.
- [4] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [5] Yuki Hou, Haruki Tamoto, and Homei Miyashita. " my agent understands me better": Integrating dynamic human-like memory recall and consolidation in llm-based agents. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–7, 2024.

- [6] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. *arXiv preprint arXiv:2408.09559*, 2024.
- [7] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, et al. Planning-oriented autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 17853–17862, 2023.
- [8] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, Li-Wei H. Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G. Mark. MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3:160035, May 2016.
- [9] Abhilasha Ashok Kumar. Semantic memory: A review of methods, models, and current challenges. *Psychonomic Bulletin & Review*, 28:40 – 80, 2020.
- [10] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, et al. Matryoshka representation learning. *Advances in Neural Information Processing Systems*, 35:30233–30249, 2022.
- [11] Yang Li, Yangyang Yu, Haohang Li, Zhi Chen, and Khaldoun Khashanah. Tradinggpt: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance. *arXiv preprint arXiv:2309.03736*, 2023.
- [12] Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv preprint arXiv:2311.08719*, 2023.
- [13] Jiageng Mao, Junjie Ye, Yuxi Qian, Marco Pavone, and Yue Wang. A language agent for autonomous driving. *arXiv preprint arXiv:2311.10813*, 2023.
- [14] Euclides Carlos Pinto Neto, Sajjad Dadkhah, Raphael Ferreira, Alireza Zohourian, Rongxing Lu, and Ali A. Ghorbani. Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors*, 23(13), 2023.
- [15] Andrew Nuxoll and John E. Laird. Extending cognitive architecture with episodic memory. In *AAAI Conference on Artificial Intelligence*, 2007.
- [16] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [17] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.
- [18] Wenqi Shi, Ran Xu, Yuchen Zhuang, Yue Yu, Jieyu Zhang, Hang Wu, Yuanda Zhu, Joyce Ho, Carl Yang, and May D Wang. Ehragent: Code empowers large language models for complex tabular reasoning on electronic health records. *arXiv preprint arXiv:2401.07128*, 2024.
- [19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- [20] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.
- [21] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [22] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

- [23] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [24] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- [25] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- [26] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, et al. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. *arXiv preprint arXiv:2406.09187*, 2024.
- [27] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- [28] Zhangyue Yin, Qiushi Sun, Qipeng Guo, Zhiyuan Zeng, Qinyuan Cheng, Xipeng Qiu, and Xuan-Jing Huang. Explicit memory learning with expectation maximization. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 16618–16635, 2024.
- [29] Ruihong Zeng, Jinyuan Fang, Siwei Liu, and Zaiqiao Meng. On the structural memory of llm agents. *arXiv preprint arXiv:2412.15266*, 2024.
- [30] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [31] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- [32] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024.

A Detailed experimental setups.

A.1 Agent details and functionality

Table 3: Details of Agents and Their Functionality

Agent Name	Task	Input	Output	Retrieve Feature	#Experiences
EHRAgents	EHR tasks	Task text query	Code	Text embedding	4
AgentDriver	Autonomous Driving	Vehicle state data	Predicted trajectory	Ego state, goal and history trajectory	1
CIC-IoT Agent	IoT intrusion detection (w/ 34 attack type)	IoT packet data	Reasoning and attack type prediction	IoT environment features	3

A.2 Detailed setup on addition, deletion, and challenge scenarios

EHRAgents For the text encoder, we use OpenAI text-embedding-3-large. For the coarse evaluator, we use the built-in LLM-generated termination signal, which reflects if LLM considers the task to be completed. For the strict evaluator, we use string matching to check if the ground-truth is contained in the generated answer. For input similarity, we take the highest cosine similarity between the text embedding of all retrieved experiences and the task. For output similarity, we use pycode_similar package⁴ to detect the code plagiarism score between the execution from the retrieved memory with the highest input similarity and current model execution. For periodic deletion, we use a period of 200 and $\alpha = 0$. For history-based deletion, we set the minimal deleting frequency to 5 and threshold β to 0.3 for the coarse evaluator and 0.7 for the strict evaluator. The combined deletion reuses all hyper-parameters above.

AgentDriver For the retrieval strategy, the original retrieval pipeline proposed by [13] consists of a two-step process: first, selecting the top-3 experiences based on vector similarity, and then prompting an LLM to choose the most relevant one. To improve reproducibility, we simplify this process to a single-step retrieval, selecting only the top-1 experience based on vector similarity. For the coarse evaluator, we employ an LLM to assess the quality of retrieved experiences. The LLM outputs either “yes” or “no” to indicate whether a given experience is a “good” experience or a “bad” one. The prompt used for this LLM-based evaluation is provided in Appendix A.4. For the strict evaluator, we measure the UniAD 3-second average L2 distance between the predicted trajectory and the ground truth. Experiences with an L2 error lower than 2.5 are added to the memory bank. For input similarity, we adhere to the computation method from the original paper. Specifically, we compute the exponential negative L2 distance between the query vector and stored experiences, using the same coefficients as in [13]. For output similarity, we compute the radial basis function (RBF) kernel between the predicted trajectory and the ground-truth trajectory. The similarity score is calculated using the following formula:

$$\text{output_similarity} = \exp(-\gamma \|\mathbf{v}_1 - \mathbf{v}_2\|^2).$$

In our experiments, we set $\gamma = 1.0$. For periodic deletion, we apply a deletion period of 300 steps with a threshold of $\alpha = 1$. For history-based deletion, we set a threshold of 5.0 for the UniAD 3-second average L2 distance. An experience is deleted if it has been retrieved at least 5 times and the mean UniAD 3-second average L2 distance across all retrievals exceeds this threshold. The combined deletion mechanism incorporates all the aforementioned hyperparameters.

CIC-IoT For the text encoder, we use OpenAI text-embedding-3-large. For the coarse evaluator, similar to AgentDriver, we employ an LLM to assess the quality of the experiences. The prompt for the CIC-IoT Agent’s LLM evaluator is provided in Appendix A.4. For the strict evaluator, we use

⁴https://github.com/fyrestone/pycode_similar.git

string matching to check if the ground-truth is contained in the generated answer. For input similarity, we use a feature-based approach. To measure input similarity, we adopt a feature-based approach. Specifically, we compute the relative change across all features and take the average. For continuous features, the relative change between two inputs, $\mathbf{input}_1$ and $\mathbf{input}_2$, for feature f_i is computed as:

$$S_{\text{cont}}(f_i) = \frac{|\mathbf{input}_1(f_i) - \mathbf{input}_2(f_i)|}{\max(|\mathbf{input}_1(f_i)|, |\mathbf{input}_2(f_i)|)}$$

For discrete features, we define the relative change as follows:

$$S_{\text{disc}}(f_i) = \begin{cases} 0, & \text{if } \mathbf{input}_1(f_i) = \mathbf{input}_2(f_i) \\ 1, & \text{otherwise} \end{cases}$$

For output similarity, we calculate the embedding similarity. For periodic deletion, we set a cycle of 300 steps, and $\alpha = 1$. For history-based deletion, we set the experience quality threshold $\beta = 0.5$. If an experience is retrieved 5 or more times and the average score of all test samples when using this experience falls below 0.5, the experience is removed. The combined deletion strategy reuses all the hyperparameters mentioned above.

A.3 CIC-IoT Agent design

Below is the user prompt used in CIC-IoT Agent to incorporate IoT packet data and all possible attack types.

User Prompt

Based on the following features, determine the most likely attack type from the list below:

Flow duration [description: Duration of the packet's flow]: {flow_duration}
Header Length [description: Header Length]: {Header_Length} bytes
Protocol Type [description: IP, UDP, TCP, IGMP, ICMP, Unknown (Integers)]: {Protocol_Type}
Duration [description: Time-to-Live (ttl)]: {Duration}
Rate [description: Rate of packet transmission in a flow]: {Rate}
Srate [description: Rate of outbound packets transmission in a flow]: {Srate}
Drate [description: Rate of inbound packets transmission in a flow]: {Drate}
Number of FIN flags [description: FIN flag value]: {fin_flag_number}
Number of SYN flags [description: SYN flag value]: {syn_flag_number}
Number of RST flags [description: RST flag value]: {rst_flag_number}
Number of PSH flags [description: PSH flag value]: {psh_flag_number}
Number of ACK flags [description: ACK flag value]: {ack_flag_number}
Number of ECE flags [description: ECE flag value]: {ece_flag_number}
Number of CWR flags [description: CWR flag value]: {cwr_flag_number}
Number of ACK packets [description: Number of packets with ACK flag set in the same flow]: {ack_count}
Number of SYN packets [description: Number of packets with SYN flag set in the same flow]: {syn_count}
Number of FIN packets [description: Number of packets with FIN flag set in the same flow]: {fin_count}
Number of URG packets [description: Number of packets with URG flag set in the same flow]: {urg_count}
Number of RST packets [description: Number of packets with RST flag set in the same flow]: {rst_count}
HTTP traffic flag [description: Indicates if the application layer protocol is HTTP]: {HTTP}
HTTPS traffic flag [description: Indicates if the application layer protocol is HTTPS]: {HTTPS}
DNS traffic flag [description: Indicates if the application layer protocol is DNS]: {DNS}

(Continued on next page...)

User Prompt

(Continuation from previous page...)

Telnet traffic flag [description: Indicates if the application layer protocol is Telnet]: {Telnet}
SMTP traffic flag [description: Indicates if the application layer protocol is SMTP]: {SMTP}
SSH traffic flag [description: Indicates if the application layer protocol is SSH]: {SSH}
IRC traffic flag [description: Indicates if the application layer protocol is IRC]: {IRC}
TCP traffic flag [description: Indicates if the transport layer protocol is TCP]: {TCP}
UDP traffic flag [description: Indicates if the transport layer protocol is UDP]: {UDP}
DHCP traffic flag [description: Indicates if the application layer protocol is DHCP]: {DHCP}
ARP traffic flag [description: Indicates if the link layer protocol is ARP]: {ARP}
ICMP traffic flag [description: Indicates if the network layer protocol is ICMP]: {ICMP}
IPv4 traffic flag [description: Indicates if the network layer protocol is IP]: {IPv}
LLC traffic flag [description: Indicates if the link layer protocol is LLC]: {LLC}
Total sum of feature values [description: Summation of packets' lengths in the flow]: {Tot_sum}
Minimum value [description: Minimum packet length in the flow]: {Min}
Maximum value [description: Maximum packet length in the flow]: {Max}
Average value [description: Average packet length in the flow]: {AVG}
Standard deviation [description: Standard deviation of packet length in the flow]: {Std}
Total size of the flow [description: Packet's length]: {Tot_size} bytes
Inter-arrival time (milliseconds) [description: The time difference with the previous packet]: {IAT}
Number of packets or flows [description: The number of packets in the flow]: {Number}
Magnitude of the flow [description: Average of the lengths of incoming packets in the flow + average of the lengths of outgoing packets in the flow]: {Magnitude}
Radius of the flow [description: Variance of the lengths of incoming packets in the flow + variance of the lengths of outgoing packets in the flow]: {Radius}
Covariance of the flow [description: Covariance of the lengths of incoming and outgoing packets]: {Covariance}
Variance of the flow [description: Variance of the lengths of incoming packets in the flow / variance of the lengths of outgoing packets in the flow]: {Variance}
Weight of the flow [description: Number of incoming packets / number of outgoing packets]: {Weight}
Attack Types:
['Backdoor_Malware', 'BrowserHijacking', 'DDoS-UDP_Flood', 'DDoS-SynonymousIP_Flood', 'Uploading_Attack', 'Mirai-udpplain', 'DDoS-SlowLoris', 'DDoS-ICMP_Flood', 'Recon-OSScan', 'DoS-HTTP_Flood', 'DNS_Spoofing', 'Benign-Traffic', 'DoS-UDP_Flood', 'DictionaryBruteForce', 'DoS-SYN_Flood', 'Recon-PortScan', 'Recon-HostDiscovery', 'DDoS-PSHACK_Flood', 'Mirai-greeth_flood', 'VulnerabilityScan', 'DDoS-RSTFINFlood', 'Recon-PingSweep', 'DDoS-UDP_Fragmentation', 'DDoS-HTTP_Flood', 'XSS', 'Mirai-greip_flood', 'DoS-TCP_Flood', 'MITM-ArpSpoofing', 'DDoS-ACK_Fragmentation', 'CommandInjection', 'SqlInjection', 'DDoS-ICMP_Fragmentation', 'DDoS-TCP_Flood', 'DDoS-SYN_Flood']

A.4 Coarse evaluator prompts

Below is the system prompt used for our coarse evaluator (LLM judge) in EHRAgent:

System Prompt

You are a helpful AI assistant. Solve tasks using your coding and language skills. In the following cases, suggest python code (in a python coding block) or shell script (in a sh coding block) for the user to execute. 1. When you need to collect info, use the code to output the info you need, for example, browse or search the web, download/read a file, print the content of a webpage or a file, get the current date/time. After sufficient info is printed and the task is ready to be solved based on your language skill, you can solve the task by yourself. 2. When you need to perform some task with code, use the code to perform the task and output the result. Finish the task smartly. Solve the task step by step if you need to. If a plan is not provided, explain your plan first. Be clear which step uses code, and which step uses your language skill. When using code, you must indicate the script type in the code block. The user cannot provide any other feedback or perform any other action beyond executing the code you suggest. The user can't modify your code. So do not suggest incomplete code which requires users to modify. Don't use a code block if it's not intended to be executed by the user. If you want the user to save the code in a file before executing it, put # filename: <filename> inside the code block as the first line. Don't include multiple code blocks in one response. Do not ask users to copy and paste the result. Instead, use 'print' function for the output when relevant. Check the execution result returned by the user. If the result indicates there is an error, fix the error and output the code again. Suggest the full code instead of partial code or code changes. If the error can't be fixed or if the task is not solved even after the code is executed successfully, analyze the problem, revisit your assumption, collect additional info you need, and think of a different approach to try. When you find an answer, verify the answer carefully. Include verifiable evidence in your response if possible. Reply "TERMINATE" in the end when everything is done.

Below is the system and user prompt used for our coarse evaluator (LLM judge) in AgentDriver:

System Prompt

You are a highly knowledgeable and rigorous judge for autonomous driving. You are judging a *short-horizon* trajectory (e.g., 6 steps). We only require the following:

- 1) The predicted trajectory should *generally* move towards or align with the goal.
- 2) It should stay within a drivable area (i.e., allowed region).
- 3) It should avoid collisions with other objects.

Your output format:

- First line: strictly output 'yes' or 'no'.

- Following lines: provide your reasoning (Chain-of-Thought is allowed).

Be mindful that small lateral or partial forward movements can be acceptable as long as the overall direction is consistent with the planning target and safety requirements. Our coordinate system is such that the x-axis is lateral, and the y-axis is forward. Therefore, moving forward means an increase in y values.

Be mindful that minor lateral adjustments or minimal forward movements are acceptable.

If the y coordinate is increasing from step to step (and there's no collision or out-of-lane), that may be considered a success.

User Prompt

Below are the relevant information for this autonomous driving task:

1) Current state of the ego vehicle:

{ego_prompts}

2) Perception of the environment:

{perception_prompts}

3) Commonsense:

{commonsense_mem}

4) Planning target:

{planning_target}

5) Predicted trajectory:

{pred_traj}

Please decide if the predicted trajectory is successful under the above criteria, then provide your reasoning (you may use chain-of-thought).

Remember:

- First line of your answer: 'yes' or 'no' ONLY.
- Following lines: your reasons or chain-of-thought.

Below is the user prompt used for our coarse evaluator (LLM judge) in CIC-IoT Agent:

User Prompt

You are an expert in identifying IoT attack categories. I will provide you with a problem and an answer that needs evaluation. Your task is to determine whether the given answer is correct or not.

Problem:

Based on the following features, determine the most likely attack type from the list below:

Features:

{problem}

Answer to be Evaluated:

{model_answer}

- Respond with your judgement and explanation as following format.
- First line: Respond with 'correct' or 'incorrect' only.
- Following lines: Provide your reasoning or chain-of-thought.

Your judgement:

B Additional results

We present additional results in this section.

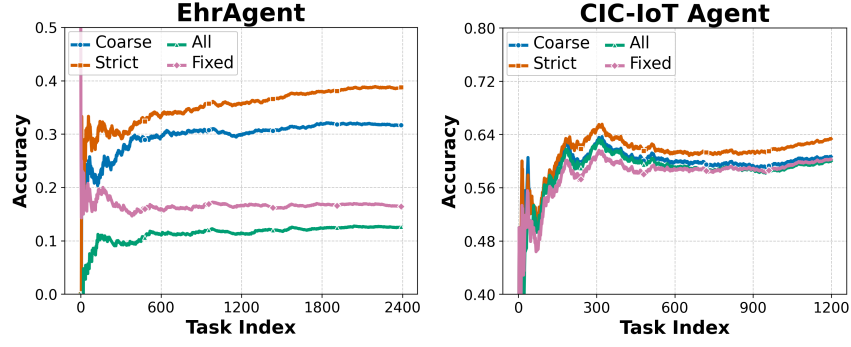


Figure 8: Accuracy trends of different addition strategies over long-term running on EhrAgent and CIC-IoT.

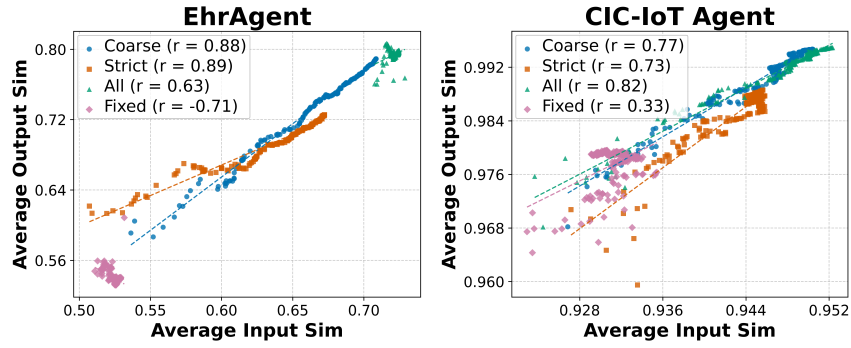


Figure 9: Cumulative average output similarity vs. input similarity of different addition strategies over the execution on AgentDriver and CIC-IoT.

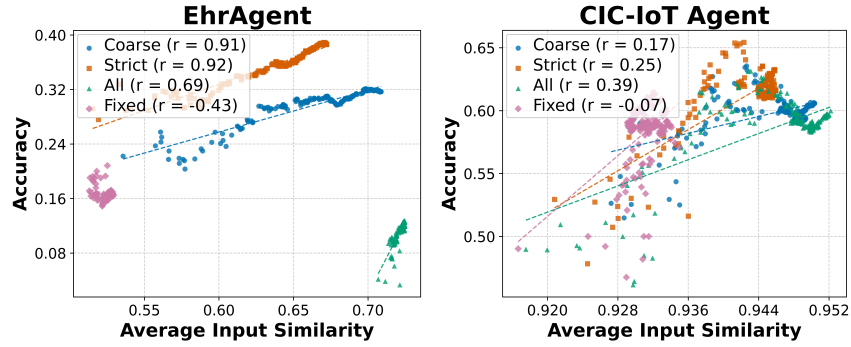


Figure 10: Cumulative average input similarity vs. performance of different addition strategies over execution.

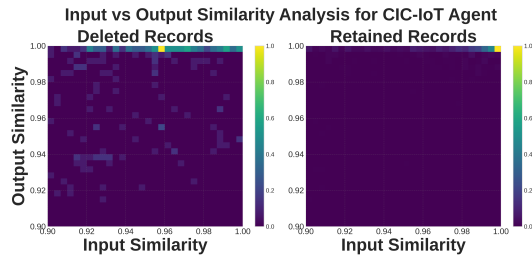


Figure 11: Addition results for misaligned memory reply on CIC-IoT Agent.

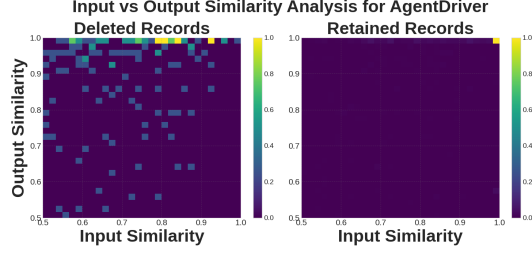


Figure 12: Addition results for misaligned memory reply on AgentDriver.

C Additional ablation results of AgentDriver

C.1 Experiments on different LLM backbones

In this experiment, we conduct experiments with different LLM Agent backbones. Specifically, we conduct evaluation on GPT-4o and Deepseek-V3 with fixed-memory baseline, strict addition, strict addition with history-based deletion, and strict addition with combined deletion in Figure 14 and Figure 13, respectively. Across these two models, we observed consistent trend within the main experiments. In addition, their input similarity versus output similarity using add strict strategies and show in Figure 15.

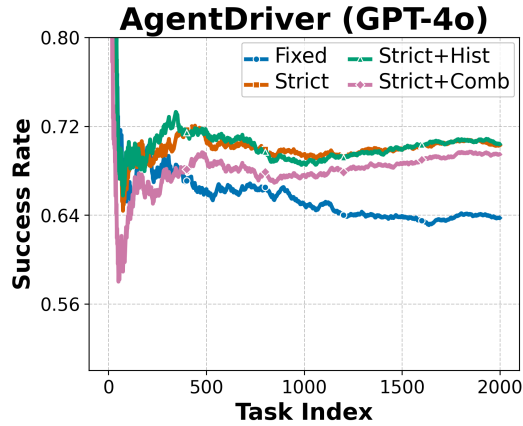


Figure 13: Accuracy trends of GPT-4o over long-term running on AgentDriver

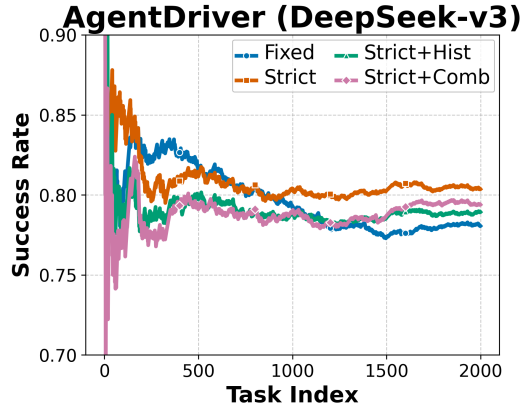


Figure 14: Accuracy trends of Deepseek-V3 over long-term running on AgentDriver

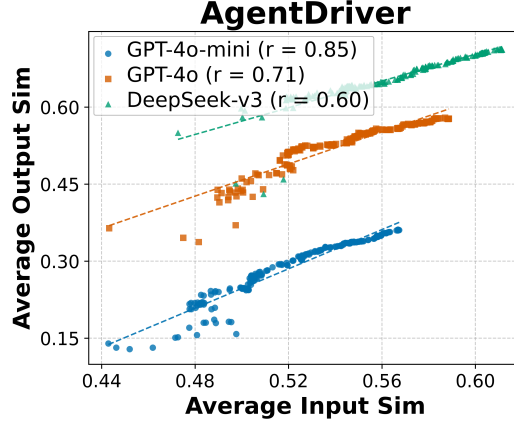


Figure 15: Cumulative average output similarity vs. input similarity over GPT-4o-mini, GPT-4o, and Deepseek-V3 using add strict in AgentDriver

C.2 Error-free variant of history-based deletion

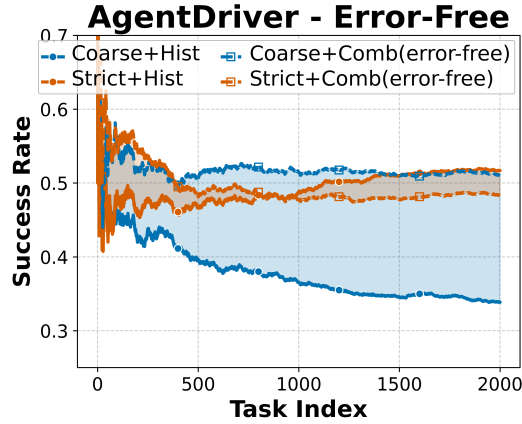


Figure 16: Comparison between history-based deletion and its error-free variants.

In Figure 16, we follow the procedure in Section 4.4 to plot the history-based deletion and its error-free variant. Surprisingly, we observe that around task index 1000, the combined deletion with strict addition surpasses the performance of its error-free variant. This suggests that history-based deletion can retain memory entries with outputs suitable for later reuse, corroborating the necessity of adopting this type of deletion.

C.3 Performance trend with noisy initialization

In this experiment, as shown in Figure 17, we initialize the memory bank in the same manner as the fixed memory bank setting described in the main experiment. Additionally, noise is introduced to the executions of the initial memories, following a normal distribution with a mean of 0 and a variance of 5.0. The experiment is conducted on a subset comprising the first 500 test samples out of the 2000 test samples used in the main experimental setup.

C.4 Performance with different fixed memory sizes

We provide additional results on memory resource constraints with different memory limitation sizes.

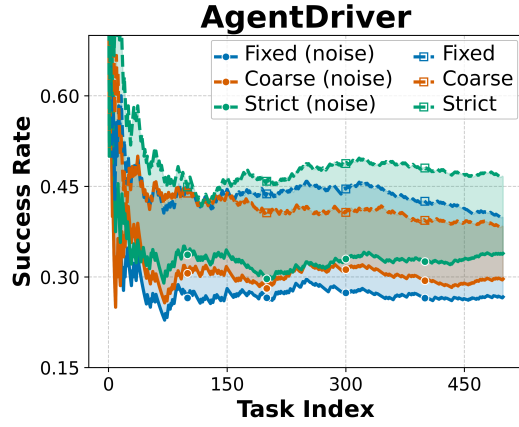


Figure 17: Average performance trend over time with initial memory that has noise with L2=5 on AgentDriver

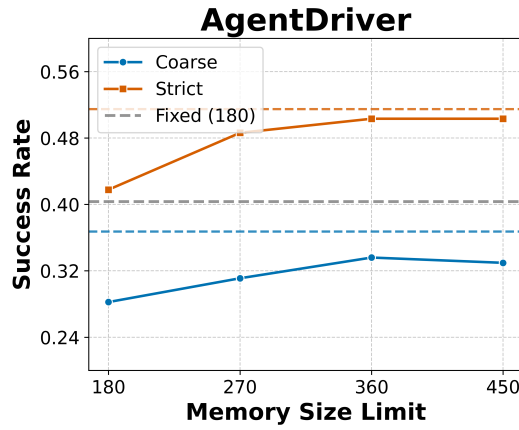


Figure 18: Different limited sizes versus the performance of AgentDriver. The horizontal dashed lines are their corresponding unlimited variant performance.